

- **Question 1**

- Step Heuristic:

$h_{step}(n) = n$ where $0 \leq n \leq 4$
and n is the total number of different
character indices between the current state
and the goal state.

- The unavoidable cost is that a character change will always require at least one action per character. In the case of the step cost, it just happens that the estimated cost will always be identical to the true cost because there is no weight or rarity applied to the operations.
 - It can never be overestimate because it uses the cheapest possible combination of character changes to turn the current state into the goal state. In this cost function, it just so happens that the heuristic and the action cost are always identical.

- Scrabble Heuristic:

$h_{scrab}(n) = \sum i_{s.cost}$ from $i = 0$ to n , where $0 \leq n \leq 4$
where n is the total number of different
character indices between the current state
and the goal state, and $i_{s.cost}$ is the scrabble
cost of each character.

- The unavoidable cost is that each character has an associated rarity or weight assigned to it such that some characters are “more expensive” to replace than others. In this circumstance, much like the step cost function and heuristic, it is once again identical to the true cost, but this time it isn’t just that we know how many steps it takes, but also the true value of every character change required in a step.
 - It can never be overestimate because it uses the cheapest possible combination of character changes to turn the current state into the goal state. The cost function also happens to be identical to the heuristic. We could have chosen to use a simple step analysis like the step heuristic. But utilizing the same function for both $f(n)$ and $h(n)$ will never produce an overestimate of the true optimal remaining cost,

it will always be equivalent, and so it still provides a technical lower bound.

- Frequency Heuristic:

$h_{freq}(n) = mn$ where $0 \leq n \leq 4$
*and n is the total number of different
character indices between the current state
and the goal state, and $m = (1 + \min_rarity(\omega))$,
representing $1 + \text{minimum rarity in dictionary}$*

- The unavoidable cost is once again that any character change will require at least a value of 1. The frequency cost still requires a cost given by the step itself. However, now we can never be truly certain which path might be taken, so we can only ever estimate based on the minimum rarity calculation and the number of steps as given by the frequency cost calculation of $1 + rarity(\omega)$
- Because of the above, the highest cost of $h(n)$ must equal the minimum cost $f(n)$, because it can never be that the heuristic rarity calculation will be greater than the minimum. This only ever occurs when it just so happens that the state change from then current state to goal state is the word “the” in a single step. However, this is altogether rare given the size of the dictionary, and as a result the difference between the heuristic estimate and the true cost will almost always be greater than 0. This means that the lower bound heuristic estimate in practice will virtually never be a perfect estimate.

- **Question 2**

Pair	Cost Measure	UCS Cost	UCS Expansions	A* Cost	A* Expansions	Reduction
why → ask	Steps	5	773	5	16	97.93%
why → ask	Scrabble	9	765	9	13	98.3%
why → ask	Freq	38.86	808	38.86	754	6.68%
bear → duck	Steps	4	691	4	6	99.13%

bear → duck	Scrabble	11	2981	11	6	99.80%
bear → duck	Freq	38.24	1084	38.24	714	34.13%

- UCS and A* always returned the optimal cost
- Word Pair Cost Measures
 - why → ask : saw the greatest reduction through the scrabble cost measurement
 - bear → duck : saw the greatest reduction through the scrabble cost measurement
 - In both circumstances, scrabble cost saw the greatest reduction in cost.
- The frequency heuristic was the least informative in both instances. It's best case was in bear → duck, but even that was only ~34% compared to the 90+% reductions seen in scrabble and step. It's reductions were even worse in the why → ask pairing.
- Differences caused by heuristic estimates
 - Step
 - The heuristic for the step estimate provides an exact lower bound to the solution, because the actual cost calculation is simple and only requires one rule and one cost → for every index of the current state, if it is not the proper character for the same index of the goal state, that letter must change and it will cost 1. Because of this, the heuristic is identical to the cost function itself, and therefore it provides an excellent lower bound estimate for A*. However, because everything has equal weight, the node exploration in the frontier is somewhat random based on how far a given current state index is from the beginning of the alphabet since my comparison dictionary is in alphabetic order. If I adjusted the dictionary I used for the alphabet comparison to be in order of the letters most frequently used in English, it would be more like the scrabble heuristics results.
 - The scrabble heuristic is the most informed. Not only do steps matter as a measure of cost, but each letter has a cost associated with the scrabble table. Because of this, A* needs to do even less expansions because the nodes generated can

be weighed based on the scrabble cost of each character. So the algorithm experiences a reduction in randomness for evaluating a node path and it's relevant cost, thus requiring fewer node expansions in the frontier to find an optimal path.

- The frequency heuristic is the least informed. Because the cost calculation itself is dependent on the **word rarity cost** instead of individual character costs, we can't make an accurate lower bound estimate of the true cost itself. This is because the difference between true cost – estimated cost will in practice always be greater than zero. This is because the best we can predict is the lowest possible rarity for any given word in the dictionary. The $\text{dictionary_min} = 0.016226212$, $\text{dictionary_max} = 16.347363$ and the $\text{rarity_average} > 1$ (and probably much closer to ~6). Since $h(n)$ is a very liberal lower estimate, it means that the actual cost – estimated cost will almost never be zero.

- **Question 3**

- Step

- The optimal cost from W_1 to W_2 will always be the same as W_2 to W_1 because the ruleset means that each character will always be converted to the exact next character, and each change is exactly the same cost, and thus the transitions are symmetrical.

- Scrabble

- The optimal cost may be different. For example, $\text{why} \rightarrow \text{cat} = 5$, but $\text{cat} \rightarrow \text{why} = 12$, this is because the cost of each letter may be different in reverse, and thus the transitions are not inherently symmetrical.

- Frequency

- Should always be the same cost, because traveling backwards based on word cost will mean that the cheapest cost in reverse should be identical to what the cheapest cost forward was. The transition in either direction should be symmetrical.

- **Question 4**

- GPT-5.6 Luna. The prompt given was the one provided in the project documentation. The proposed heuristic is similar to the one I've implemented with one caveat: for the total number of characters which need to be changed, sum the equivalent number of minimum rarities and add them by the total number of characters. That is to say:

$$h(n) = n + \sum r_i \text{ where } r_i = r_1, r_2, r_3, \dots, r_m \text{ and correlate}$$
to the sequentially cheapest rarity word in the dictionary
starting from the lowest rarity cost word, and
n = total number of necessary character swaps.

The heuristic is admissible, and better than the one I came up with, because reasonably, if you have to change 3 characters (just as an example), then at best the 3 changes that you can make would produce the 3 cheapest-cost-rarity words in the dictionary. This would provide a better lower bound than simply calculating for the lowest cost word once, because it approaches a closer lower-bound estimate to the real cost than my function does.

Pair	Cost Measure	UCS Cost	UCS expansions	A* Cost	A* Expansions	Reduction	Original Reduction
why → ask	Freq	38.86	808	38.86	748	7.43%	6.68%
bear → duck	Freq	38.24	1084	38.24	681	37.18%	34.13%

AI USAGE STATEMENT:

The predominant use of AI was by giving chatgpt the totality of the dcsolve.py and search.py files to understand the underlying abstraction of how I was supposed to implement my dc.py file. The issue I had was trying to parse through all of the code and identify exactly how my implementation interacted with the given search.py file.

I also frequently had to give my code to gpt to inform me of proper syntax usage in python because I haven't touched it in 2 years. Effectively I treated it as an intelligent API dictionary, asking for function calls within the default library to achieve the goal I was trying to reach. For example, unpacking tuples and how to properly concatenate characters in a string to generate a new word. I forgot that strings in python are immutable, and so the solution to building new strings is very different than the c approach.

I also used it to help fix the dctest.py implementation I tried to make in order to answer question 2. My initial test code was completely wrong and did not work, so I defaulted to

the example provided by chatgpt which I generated using the project requirements documentation and the project code. I reviewed the given solution and applied it.

Lastly I used it as a sanity check for my understanding of the problems I was attempting to answer and solve.

Generally my use-case ruleset is to only ask a question that are as specific as possible, and not ask for the solution itself, but whether my understanding is accurate or flawed and why. The math calculations are all my machinations, as well as all of the code in `dctest.py` except for the function `AI_h_frequency_calculation()`, which is the implementation for the AI proposed heuristic. I kept getting incorrect summations for the dictionary because I couldn't figure out how to properly manipulate the rarities list iteratively from `sorted(dictionary.values())`.